CampusIQ - Day 3 Notes

Name: Gufran Date: 20 October 2026 Role: AI (ai folder)

Topic: Embeddings, cosine similarity, semantic search, score threshold

0. The problem from Day 2

- Keyword search needs the same words. "presence" and "attendance" mean the same, but the code saw different words, so every score was 0.
- Fix: turn text into numbers that capture meaning. These numbers are called an embedding (a vector).

1. What is an embedding?

- An embedding is a list of numbers that represents a piece of text. Similar meaning gives numbers that are close together.
- A small AI model creates it. It learned from huge amounts of text which words and sentences mean similar things.
- Toy example in 2 numbers (real ones have hundreds):

```
"attendance"  -> [0.90, 0.10]
"presence"    -> [0.85, 0.15]     # very close to attendance
"hostel fee"  -> [0.10, 0.95]     # far away
```

- Our model `all-MiniLM-L6-v2` gives 384 numbers per text (size is about 90 MB). Later we will use BGE-M3 with 1024 numbers.
- Cosine similarity measures how close two vectors are by their angle. Near 1 = same meaning. Near 0 = unrelated.

2. How semantic search works

1. Embed every chunk **once** and keep the vectors.
2. Embed the question.
3. Compare the question vector with every chunk vector (cosine similarity).
4. Sort by score and return the top few chunks.

3. Installing sentence-transformers

```
pip install sentence-transformers
python -c "import sentence_transformers; print('ready')"
```

- It also installs **PyTorch** (the engine for AI models), so it is a big download. Version used: sentence-transformers 6.1.0, torch 2.14.1.
- After "Installing collected packages" the terminal shows nothing for 3–8 minutes. It is not frozen. Do not press Ctrl+C.
- My mistake: I cancelled it once ("Operation cancelled by user"). Fix: run the same command again. Cached files make it faster.
- First import takes 10–60 seconds. The first model download is about 90 MB. Warning about "HF Hub token" can be ignored.

4. First embedding

```
from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer("all-MiniLM-L6-v2")
emb = model.encode(sentences)       # 1 vector per sentence
print(emb.shape)                     # (3, 384)
util.cos_sim(emb[0], emb[1])         # similarity of 2
```

Pair of sentences	Score
attendance for exam vs presence for finals	0.521 (same meaning, no shared words)
attendance for exam vs hostel fee date	0.362
presence for finals vs hostel fee date	0.207

- Do not trust the absolute number. Trust the **ranking**. Two questions written in a similar style can score higher than expected.

5. Semantic search over my 9 chunks

```
chunk_vectors = model.encode(chunk_texts)       # once, at start

def search(question, top_k=3):
    q = model.encode(question)
    scores = util.cos_sim(q, chunk_vectors)[0]
    ranked = sorted(zip(scores.tolist(), chunks),
                    key=lambda pair: pair[0], reverse=True)
    good = [(s, c) for s, c in ranked if s >= MIN_SCORE]
    return good[:top_k]
```

- `zip(a, b)` pairs items of two lists. `sorted(..., reverse=True)` puts the best first.
- `[(s, c) for s, c in ranked if s >= MIN_SCORE]` is a **list comprehension**: a loop and a filter in one line.

5b. Test results

Question	Best match	Result
Minimum presence to sit in finals?	75 percent attendance rule (0.491)	Correct. Keyword search gave 0 for all.
How many marks to pass?	40 percent overall (0.655)	Correct.
What time back in hostel?	return by 9 PM (0.681)	Correct.
When can visitors come to hostel?	#1 was "return by 9 PM" (0.678). Visitors rule was #2 (0.591).	Near miss. Right chunk is in top 3.
What if my attendance is 70 percent?	Right rule ("between 65 and 75") came #3 (0.425).	Embeddings cannot do arithmetic. All 3 attendance rules were returned.

Lessons: (1) Give the LLM the top 3-5 chunks, not only the first one. It can reason over them and do the maths. (2) Our architecture has a **reranker** stage for exactly this: embedding search finds candidates quickly, then a reranker re-reads them carefully and fixes the order.

5c. Small mistakes today

- Pressed cancel during the pip install, then re-ran it.
- Used `cd ..` when I was already in the right folder. Always read the end of the prompt first.
- Ignored two harmless messages: the VS Code venv popup and the HF Hub warning.

6. Score threshold: "I don't Know"

- Question: "What is the menu in the canteen today?" There is no answer in the rules. Top score was only 0.223.
- Real matches scored **0.49 to 0.70**. So we set `MIN_SCORE = 0.30` and ignore anything below.
- If nothing passes, print `No relevant rule found.` Later the AI will say "I could not find this in the regulations" instead of making up an answer (hallucination).
- Caution: 0.30 comes from only 9 chunks and a few questions. The real value will be tuned on a **golden test set of 100 questions.**

7. Keyword vs embedding search

	Keyword (Day 2)	Embedding (Day 3)
Matches	same words	meaning
Synonyms ("presence")	fails	works
Exact terms (roll no, codes)	works well	can be weak
Numbers / arithmetic	no	no
Speed	very fast	fast after vectors are stored

- That is why the final system uses **hybrid search**: BM25 (Keyword) + embeddings, merged with RRF, then reranked.

9. Self-check (answers)

- What is an embedding? A list of numbers that represents the meaning of text. Similar meanings have close vectors.
- Why did Keyword search fail? It only matches identical words.
- What does cosine similarity measure? How close two vectors are (angle). Near 1 = similar.
- Why return top 3-5 chunks, not just 1? The best chunk is not always #1, and the LLM can combine several.
- Why a minimum score? To say "not found" instead of guessing wrong.

10. Quick revision sheet

```
MODEL    SentenceTransformer("all-MiniLM-L6-v2")  # 384 nums
VECTORS  v = model.encode(list_of_texts)    # (n, 384)
SCORE    util.cos_sim(a, b)                 # about 0 to 1
SEARCH   embed chunks once, embed question, cos_sim, sort
FILTER   keep only score >= MIN_SCORE (0.30)
GIT      repo root: pull, add ., commit -m "", push
```

11. Files created today

- `ai/day3_embed.py` - first embeddings and similarity
- `ai/day3_search.py` - semantic search with a threshold

Day 3 complete. Next (Day 4): read a real PDF with PyMuPDF and chunk it with overlap, so no rule is cut in half.